Projektuppgift

Frontend-baserad webbutveckling

Ziad Adeleo

Mittuniversitetet

MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. **Campus Sundsvall** Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00, Fax: +46 (0)771 97 50 01.

MITTUNIVERSITETET
Institutionen för data och elektroteknik(DET)

Författare: Ziad Adeleo, ziba2500@student.miun.se
Utbildningsprogram: Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: VT, 2026

Sammanfattning

Country Currency Converter är en Single Page Application som visar aktuella och historiska växelkurser mellan länder med hjälp av REST Countries, Frankfurter API och Chart.js. Användaren kan mata in ett belopp, välja två länder och se värde i en annan valuta och en tioårs historik i linjediagram.

Applikationen implementerar felhantering, ARIA-attribut, tabb-navigering och en dold tabell för skärmläsare. Spinner visas innan diagrammet laddas, och logotypen är animerad för att förstärka visuell identitet.

Innehållsförteckning

Sammanfattning **iii**

1 Introduktion **5**
1.1 Bakgrund och problemmotivering 5
1.2 Övergripande syfte 5
1.3 Avgränsningar 6
1.4 Detaljerad problemformulering 6

2 Teori **6**
2.1 Utvecklingsmiljö 6
2.2 Frontend teknologier 7
2.3 API 7
2.4 Datavisualisering 8
2.5 Webbanvändbarhet och tillgänglighet 8
2.6 Automatisering 9
2.7 Byggverktyg 9
2.8 Testning 9

3 Metod **11**

3.1 Research 11

3.2 Git och gitHub 11

3.3 Design och arkitektur 11

3.4 Bedömning och felhantering 12

3.5 Struktur och stil 12

3.6 Logik 12

3.7 Testning 12

3.8 Publicering på Render 13

4 Konstruktion **13**

5 Resultat **14**

6 Slutsatser **14**

Källförteckning **15**

1 Introduktion

Country Currency Converter är en Single Page Application som har utvecklats med hjälp av HTML, SCSS och JavaScript. Applikationen använder externa API:er, Restcountries och Frankfurter API, för att hämta land- och växelkurs. För visualisering av historiska växelkurser används Chart.js, och projektet har byggts med hjälp av Vite som byggverktyg.

Webbapplikationen visar hur mycket ett angivet belopp i ett lands valuta motsvarar i ett annat lands valuta, exempelvis 100 SEK = 11,07 USD. Utöver den aktuella växelkursen presenteras även ett linjediagram med historisk data som visar hur värdet av en enhet i en valuta har förändrats i förhållande till en annan valuta under en tioårsperiod.

Applikationen har utvecklats med fokus på tillgänglighet och användbarhet. Den är utformad för att kunna användas både av personer utan funktionsnedsättning och av personer med funktionsnedsättning. För att öka tillgängligheten har ARIA-attribut implementerats på HTML-element. En tabell som inte syns. Tabellen motsvarar informationen i linjediagrammet har inkluderats för användare som inte kan ta del av grafisk information. Attributet tabindex="0" har även använts för att förbättra navigeringen för användare med motoriska svårigheter.

1.1 Bakgrund och problemmotivering

Många webbapplikationer för valutakonvertering presenterar främst aktuella växelkurser och identifierar valutor genom flaggor och valutatecken. Kopplingen till respektive land framgår inte alltid tydligt för användaren. Det visas heller inte ett längre historiskt intervall som gör det möjligt att analysera hur växelkursen har förändrats över tid.

Grafisk presentation av växelkurser kan bidra till en ökad förståelse för hur stark eller svag en valuta är i förhållande till en annan. När värdet av en valutas enhet presenteras i ett linjediagram blir förändringar över tid mer tydliga och lättare att tolka. Sådan information kan utgöra ett grundunderlag för beslut, exempelvis vid investeringar i ett annat land.

Det finns därför ett behov av en webbapplikation som kombinerar tydlig identifiering av länder och valutor med visualisering av historisk data över en längre tidsperiod.

1.2 Övergripande syfte

Syftet med webbapplikationen Country Currency Converter är att ge användare information om hur två länders valutor förhåller sig till varandra i nuläget och över en tioårsperiod.

1.3 Avgränsningar

Målet med applikationen var att implementera funktionalitet för valutakonvertering och visualisering. Detta inkluderar bland annat att användaren kan mata in heltal och flyttal med upp till två decimaler i inmatningsfältet för belopp (amount), felhantering vid försök att hämta data för två identiska valutor, felmeddelanden om valutapar eller diagram inte kan laddas eller om hela applikationen kraschar. Responsivitet har testats på olika webbläsare för att säkerställa att gränssnittet anpassar sig korrekt.

Applikationen har inga omfattande användbarhetstester och tillgänglighetstester. Även om Windows 11 inbyggda skärmläsare Narrator har använts för att kontrollera att ARIA-attributen fungerar, är detta inte tillräckligt för att garantera full tillgänglighet för alla användare med funktionsnedsättningar.

1.4 Detaljerad problemformulering

Många webbapplikationer för valutakonvertering visar endast aktuella växelkurser och erbjuder begränsad visualisering av historisk data. Valutor identifieras ofta genom flaggor och valutatecken, vilket gör det svårare för användaren att tydligt koppla valuta till land. Bristen på historisk data och grafisk presentation gör det svårt att tolka hur valuta har utvecklats över tid. Detta leder till att användare inte får en fullständig bild av valutans styrka eller svaghet, vilket kan påverka beslut vid investeringar.

För att åtgärda dessa problem behöver applikationen kunna visa både aktuell och historisk växelkurs, visualisera data i linjediagram, ge tydlig koppling mellan valuta och land och vara tillgänglig för användare med olika typer av funktionsnedsättningar.

2 Teori

I detta avsnitt presenteras de tekniska begrepp och verktyg som ligger till grund för utvecklingen av webbapplikationen Country Currency Converter. Avsnittet behandlar utvecklingsmiljö, frontend-teknologier, APIer, datavisualisering, webbtillgänglighet, versionshantering, byggverktyg samt testning.

2.1 Utvecklingsmiljö

Visual Studio Code är en integrerad utvecklingsmiljö (IDE) som används för att skriva, organisera och hantera källkod. I utvecklingsmiljön skapas projektets mappstruktur och filer med olika filändelser beroende på programmeringsspråk,

exempelvis .html, .scss och .js.

Visual Studio Code erbjuder även tillägg (extensions) som underlättar utvecklingsprocessen genom exempelvis syntaxmarkering.

I utvecklingen av Country Currency Converter har Visual Studio Code använts för att strukturera projektet och skriva den kod som utgör applikationens struktur, stil och logik [1].

2.2 Frontend teknologier

Frontend-teknologier är en del av en webbapplikation som användaren interagerar med direkt i webbläsaren. I detta projekt används HTML, SCSS och JavaScript.

HTML (HyperText Markup Language) används för att strukturera innehållet på webbsidan. Genom olika element och taggar definieras exempelvis rubriker, formulärfält, knappar och behållare för diagram. I Country Currency Converter används HTML för att bygga upp gränssnittet där användaren anger belopp och väljer namn på länder.

SCSS (Sassy Cascading Style Sheets) är en utökad version av CSS som används för att ange utseendet på HTML-element. SCSS möjliggör användning av variabler och strukturerad kod, vilket förenklar hantering av färger, typografi och layout. I applikationen används SCSS för att skapa en konsekvent design och säkerställa god kontrast och läsbarhet.

JavaScript är ett programmeringsspråk som används för att skapa interaktivitet och dynamiskt innehåll. Det möjliggör exempelvis hämtning av data från externa källor, uppdatering av innehåll utan att sidan laddas om samt hantering av användarens inmatningar. I Country Currency Converter används JavaScript för att hämta namn på länder, deras valutatecken, valutakurser, genomföra konverteringar och uppdatera diagrammet dynamiskt [2].

2.3 API

Ett Application Programming Interface (API) är en uppsättning regler som möjliggör kommunikation mellan olika programvaror. Genom APIer kan en applikation hämta eller skicka data utan att själv behöva lagra eller bearbeta all information lokalt.

I detta projekt används två REST-baserade APIer. REST (Representational State Transfer) är en princip för hur data kan överföras via webben, vanligtvis i JSON-format.

REST Countries API används för att hämta information om länder och deras tillhörande valutatecken. Frankfurter API används för att hämta aktuella och historiska valutakurser. Genom JavaScript skickas förfrågningar till dessa API:er och den mottagna datan integreras i applikationens gränssnitt.

Detta möjliggör att applikationen kan visa uppdaterad valutadata utan att själv behöva lagra eller underhålla en egen databas [3].

2.4 Datavisualisering

Datavisualisering innebär att information presenteras grafiskt för att göra det lättare att förstå och tolka.

Chart.js är ett JavaScript-bibliotek som möjliggör skapande av interaktiva diagram direkt i webbläsaren. Biblioteket erbjuder färdiga diagramtyper som kan anpassas efter behov.

I Country Currency Converter används Chart.js för att visualisera hur värdet av en enhet i en valuta har förändrats i förhållande till en annan under en tioårsperiod. Genom den grafiska presentationen blir valutans utveckling över tid mer tydlig jämfört med enbart numeriska värden i tabellform [4].

2.5 Webbanvändbarhet och tillgänglighet

Webbanvändbarhet avser hur enkelt och effektivt en användare kan använda en webbapplikation. Tillgänglighet innebär att webbapplikationen ska kunna användas av personer med olika typer av funktionsnedsättningar.

Riktlinjer för webbtillgänglighet definieras bland annat i WCAG (Web Content Accessibility Guidelines). Dessa riktlinjer behandlar exempelvis kontrastförhållanden, tangentbordsnavigering och anpassning för skärmläsare.

För att öka tillgängligheten i Country Currency Converter har följande implementerats:

- Kontroll av kontrastförhållanden enligt WCAG
- Användning av ARIA-attribut för att förbättra tolkning av innehåll med skärmläsare
- Implementering av tabindex för tangentbordsnavigering
- En “dold” tabell som representerar linjediagrammets data för användare som inte kan ta del av grafisk information [5].

2.6 Automatisering

Git är ett versionshanteringssystem som används för att spåra förändringar i källkod över tid. Det möjliggör att tidigare versioner av projektet kan återställas och att utvecklingen dokumenteras.

GitHub är en plattform som bygger på Git och används för att lagra och hantera projektets källkod. Genom att arbeta med en lokal Git-repository och en fjärrrepository på GitHub har förändringar i källkoden kunnat versionshanteras och säkerhetskopieras över tid.

Render är en molnbaserad tjänst för hosting av webbapplikationer. Genom att koppla GitHub-repositoryt till Render uppdateras applikationen automatiskt varje gång en ny version publiceras till huvudgrenen main på GitHub. Detta möjliggör kontinuerlig uppdatering av webbapplikationen och gör den uppdaterade applikationen tillgänglig för användare via internet [6,7,8].

2.7 Byggverktyg

Vite.js är ett modernt byggverktyg för frontend-utveckling. Det fungerar både som utvecklingsserver och som verktyg för att optimera projektets filer inför produktion. Under utvecklingsfasen möjliggör Vite snabb uppdatering av förändringar i källkoden. Vid produktionsbyggnad sammanfogas och optimeras JavaScript- och CSS-filer, vilket minskar filstorlek och förbättrar laddningstid.

I Country Currency Converter används Vite för att effektivisera utvecklingsprocessen och för att optimera applikationens prestanda inför produktion samt publicering på Render [9].

2.8 Testning

En webbläsare är ett datorprogram som tolkar och kör HTML-, SCSS- och JavaScript-kod och presenterar resultatet för användaren. Eftersom olika webbläsare kan tolka webbsidor på olika sätt är det viktigt att testa applikationen i flera webbläsare för att säkerställa funktionalitet fungerar lika.

Under utvecklingsfasen har Country Currency Converter främst testats i Google Chrome. För att testa responsivitet och kompatibilitet har applikationen även testats i Microsoft Edge, Mozilla Firefox och Safari via Browserling. Tillgängligheten har kontrollerats med skärmläsaren Narrator i Windows 11 för att säkerställa att innehållet fungerar för användare med skärmläsare.

Prestanda har analyserats med Google Lighthouse både i utvecklingsmiljö och i produktionsmiljö. I utvecklingsmiljö uppmättes ett prestandavärde på 56. Efter optimering med vite.js genom kommandot npm run build samt publicering på Render uppmättes maxresultat på samtliga parametrar[10].

3 Metod

I detta avsnitt beskrivs den strukturerade arbetsprocess som användes vid utvecklingen av Country Currency Converter. Processen följde ett iterativt arbetssätt, vilket innebär att stegen 3.3 till 3.7 upprepades och utvärderades under hela utvecklingsfasen.

3.1 Research

Ett tiotal webbaserade valutakonverterare undersöktes och två saker identifierades. För det första saknades en tydlig koppling mellan landets namn och dess valuta. För det andra visades inte någon längre historik över valutakurserna. Dessa observationer låg till grund för idén att utveckla en single page applikation som både visar värdet av ett inmatat belopp i en annan valuta och presenterar en grafisk 10-års historik över valutaparen i ett linjediagram. Applikationen fick det beskrivande namnet Country Currency Converter.

3.2 Git och gitHub

Ett lokalt Git-repository initierades och kopplades till ett fjärrrepository på GitHub för att möjliggöra versionshantering av källkoden. Arbetsflödet följde mestadels principen att versionshantera en komponent i taget för att underlätta framtida förståelse eller förändring av kod. Först versionhanterades applikationens HTML och SCSS efter att layouten var responsiv. Därefter versionhanterades JavaScript-logiken, strukturerad utifrån användarens interaktion och förmåga (user capability), vilket gjorde det möjligt att följa och dokumentera utvecklingen stegvis.

3.3 Design och arkitektur

Designen för applikationen bestämdes till en single-page application och den övergripande strukturen skissades på papper för att skapa en visuell mall. Färgtemat anpassades efter användarens systeminställningar, med stöd för både light mode och dark mode. För att säkerställa god tillgänglighet definierades färgvariabler för olika element och kontrastförhållanden kontrollerades mot WCAGs riktlinjer. Det beslutades även att logotypen skulle animeras, och att sektionen för linjediagrammet skulle visa en spinner tills diagrammet färdigladdats.

Applikationens funktionalitet analyserades element för element utifrån användarens perspektiv. För varje element noterades vad användaren kunde göra och hur dessa skulle hanteras inom applikationen. Denna analys, tillsammans med val av teknologier, låg till grund för projektets modulära mappstruktur, där varje modul hanterar applikationens delar eller användarens interaktioner.

3.4 Bedömning och felhantering

I detta steg identifierades och kategoriserades potentiella fel som kan uppstå under användning av applikationen. Fel definierades på tre nivåer: kritiska applikationsfel, icke-kritiska applikationsfel och användarfel.

- Kritiska applikationsfel är sådana som gör att applikationen inte kan fungera alls, exempelvis om ett API-anrop misslyckas.

- Icke-kritiska applikationsfel påverkar endast enskilda sektioner som är beroende av API-data, utan att störa övrig funktionalitet.

- Användarfel uppstår när användaren försöker utföra handlingar som inte är tillåtna, exempelvis att söka efter två länder med samma valuta.

För varje feltyp bestämdes hur applikationen skulle hantera situationen. För att användaren ska veta vad som är fel, visas felmeddelanden i applikationen för att informera användaren om felet.

3.5 Struktur och stil

Applikationen strukturerades med semantisk HTML för att tydligt separera innehåll. Innehållet delades upp i sektioner för Header, Main och Footer.

SCSS användes och organiserades i partials med en huvudfil som importerar alla partials, för bättre struktur och återanvändbarhet. Responsiv design implementerades med flexbox och media queries för att säkerställa fungerande layout på olika skärmstorlekar.

3.6 Logik

Applikationens logik byggdes med ES6-moduler för att separera ansvar, API-anrop, databehandling och DOM-manipulation. Detta ökade kodens läsbarhet och underhållbarhet.

Diagrammet implementerades med Chart.js och uppdateras dynamiskt utan omladdning av sidan.

3.7 Testning

Testning genomfördes löpande under utvecklingen. Funktionalitet och felhantering kontrollerades genom olika användarscenarier, felhantering och simulerade API-fel.

Responsivitet testades i webbläsarens utvecklarverktyg genom olika skärmstorlekar. Lighthouse för att få en uppfattning om applikationens prestanda och hur den har implementerats. Narrator användes för att testa hur applikationen läses upp för personer med funktionshinder.

3.8 Publicering på Render

En produktionsversion skapades med Vite och publicerades via Render. GitHub-repositoriet kopplades till Render för att möjliggöra automatisk publicering vid uppdateringar.

4 Konstruktion

Konstruktionen av Country Currency Converter bygger på den metod som beskrivs i föregående kapitel. Applikationen är en Single Page Application där main.js fungerar som ingångspunkt och importerar alla ES6-moduler samt main.scss som i sin tur importerar SCSS-partials. Den modulära uppdelningen gör att varje modul hanterar ett specifikt ansvar, såsom API-anrop, databehandling eller DOM-manipulation, vilket förenklar underhåll av applikationen och felhantering.

Land- och valutadata hämtas via REST Countries API, och växelkurser samt historiska data hämtas från Frankfurter API. Dataflödet hanteras asynkront med async/await och fel från API-moduler re-throwas till main.js för en samlingspunkt för felhantering. Om ett kritiskt fel uppstår, exempelvis att API-anrop misslyckas, stoppas hela applikationen och ett felmeddelande visas.

Beloppsfältet tillåter endast heltal eller decimaltal med upp till två decimaler. Vid försök att välja två länder med samma valuta visas ett felmeddelande. Resultatet presenteras dynamiskt, exempelvis 100 SEK = 10 USD och uppdateras utan omladdning av sidan. Felmeddelanden och uppdateringar är tillgänglighetsanpassade med ARIA-attribut (role="alert" för fel, role="status" för resultat) och tabindex="0" för fokuserbar navigering. För skärmläsare finns även en dold tabell som uppdateras dynamiskt och representerar data från linjediagrammet.

Linjediagrammet implementerades med Chart.js och visar hur en valuta förhåller sig till en annan under ett tio års intervall. En spinner visas innan diagrammet är färdigt för att ge användaren visuell feedback om att data hämtas och bearbetas. När diagrammet har laddats färdigt försvinner spinnern och diagrammet visas.Vid fel visas felmeddelande i diagramsektionen.

Logotypen i header innehåller ett animerat dollartecken. Animationen ger en visuell indikator på applikationens syfte. Den påverkar inte funktionalitet eller tillgänglighet, utan är främst estetisk och hjälper användaren att associera applikationen med valutakonvertering.

Färgtema och kontrast kontrollerades enligt WCAG-riktlinjer, och applikationen har stöd för både light och dark mode.

5 Resultat

Resultatet av utvecklingen av Country Currency Converter visar att applikationen uppfyller de mål som sattes upp.

Applikationen konverterar korrekt mellan valutor baserat på användarens val av länder och inmatat belopp. Beloppsfältet tillåter endast heltal eller decimaltal med upp till två decimaler. Om användaren väljer två länder med samma valuta visas ett felmeddelande. Vid kritiska fel, till exempel om API-anrop till REST Countries med kommunikation till Frankfurter API misslyckas, döljs applikationen och felmeddelande visas. Icke-kritiska fel såsom problem med att ladda diagramdata, hanteras i applikationen och ett relevant felmeddelande visas för användaren.

Historiska växelkurser visas i ett linjediagram som uppdateras dynamiskt med Chart.js. En spinner visas medan data hämtas och diagrammet genereras. När diagrammet är klart försvinner spinnern och diagrammet visas. För skärmläsare används en dold tabell som motsvarar diagrammets data.

Logotypen i header innehåller ett animerat dollartecken som är till för att stärka applikationens identitet. Färgtema och kontraster följer WCAG-riktlinjer och applikationen fungerar i både light och dark mode utefter användarens systempreferenser. Tabindex och ARIA-attribut används för en fungerande navigering och tillgänglighet för personer med funktionsnedsättning.

Responsivitet har testats på olika skärmstorlekar, och applikationen anpassar sig korrekt för olika skärmstorlekar. Prestanda har analyserats med Lighthouse och i produktion visas toppresultaten för samtliga parametrar.

6 Slutsatser

Utvecklingen av Country Currency Converter har resulterat i en applikation som uppfyller de mål som sattes upp i projektet. Applikationen konverterar valutor korrekt, visar historiska växelkurser i ett linjediagram och hanterar olika typer av fel. Tillgänglighetsanpassningar, såsom ARIA-attribut, tabb-navigering och en dold tabell för diagramdata, har implementerats för att underlätta användning för personer med funktionsnedsättning. Responsiv design och färgtema som följer WCAG-riktlinjer säkerställer att applikationen fungerar på olika enheter och är visuellt tillgänglig.

Det finns begränsningar i projektet. Användartester har inte genomförts, vilket innebär att det inte finns stöd för hur riktiga användare upplever applikationen, hur tydlig informationen uppfattas, eller hur felmeddelanden uppfattas av användare. Även om applikationens syfte är att ge en överblick över valutakurser baserat på två länder och kunna ge stöd för investeringsbeslut, ger den endast information om historiska och aktuella växelkurser. Den tar inte hänsyn till andra faktorer som påverkar valutors styrka som ekonomiska indikatorer, politiska händelser eller inflation. För att applikationen ska ge ett mer fullständigt underlag behövs mer förklaringar på valutornas förhållande.

Källförteckning

[1] Visual studio Code, ”The open source AI code editor” https://code.visualstudio.com/ Publicerad datum saknas. Hämtad 2026-02-28.

[2] Denni Desuza, ”HTML,CSS & JavaScript: What Are They And Why Do They Matter?”, https://dev.to/denni25/html-css-javascript-what-are-they-and-why-do-they-matter-1m5f Publicerad 2025-04-15. Hämtad 2026-02-28.

[3] Michael Goodwin, ”What is an API (Application Programming Interface), *https://www.ibm.com/think/topics/api* Publicerad datum saknas. Hämtad 2026-02-28.

[4] Chart.js , “Chart.js” https://www.chartjs.org/docs/latest/ Publicerad datum saknas. Hämtad 2026-02-28.

[5] W3C, “Introduction to Web Accessibility" https://www.w3.org/WAI/fundamentals/accessibility-intro/ Publicerad datum saknas. Hämtad 2026-03-01.

[6] Microsoft, “What is Git” https://learn.microsoft.com/en-us/devops/develop/git/what-is-git Publicerad datum saknas. Hämtad 2026-03-01.

[7] Coursera, “What is GitHub?Use Cases and FAQ. https://www.coursera.org/articles/what-is-github Publicerad 2025-09-16. Hämtad 2026-03-01.

[8] App generator, ”Getting started With Rendert”. *https://app-generator.dev/docs/deployment/render/index.html* Publicerad datum saknas. Hämtad 2026-03-01.

[9] Teiym Asobo, “Vite.js: The lightweight and Lightning-fast Build Tool for Your Next Web Project” https://dev.to/teyim/vite-js-the-lightweight-and-lightning-fast-build-tool-for-your-next-web-project-541i Publicerad 2024-01-29. Hämtad 2026-03-01.

[10] Internetstiftelsen, “Vad är Webbläsare?” *https://internetkunskap.se/artiklar/ordlista/webblasare/* Publicerad 2024-01-29. Hämtad 2026-03-02.